



Rushikesh Kadepurkar

COMPUTER SCIENCE ENGINEERING · COEP TECHNOLOGICAL UNIVERSITY, PUNE

Pune, India 411005 +91 93568 66093 rushikeshk2005@gmail.com [Github](#) [LinkedIn](#)

Final-year CSE undergraduate at COEP Technological University with strong expertise in systems programming, full-stack web development, and AI engineering. Experienced in building end-to-end solutions spanning low-level C/C++ systems, containerized cloud-deployed applications, and LLM-based agentic pipelines. Seeking software engineering or research roles to build high-impact, production-grade systems.

TECHNICAL SKILLS

Languages	C/C++, Python, JavaScript, Java
Frameworks	React.js, Node.js, Express.js, Flask, LangChain, LangGraph, CrewAI, PyTorch
AI / ML	LLMs, RAG, Agentic AI, RAG, NLP, SentenceTransformers, Fine-Tuning
Infra & Tools	Docker, Redis, MySQL, MongoDB, ChromaDB, AWS, Git

EXPERIENCE

Summer Research Intern — AI/LLM Systems

Jun – Aug 2025

Indian Institute of Technology, Jammu

- Designed and fine-tuned intelligent agents and AI automation workflows for production LLM-based applications.
- Built and deployed scalable LLM solutions with a focus on agentic reasoning pipelines and performance optimization.

LLMs · Agentic AI · Fine-Tuning · AI Automation · Python

PROJECTS

NAT Traversal — UDP Hole Punching over QUIC

Feb – Apr 2026

Python · aioquic · pybind11 · C++ · UDP · Docker · TLS 1.3

- Built a peer-to-peer NAT traversal system supporting all 4 NAT types (Full Cone, Restricted Cone, Port-Restricted, Symmetric) with a STUN-based classifier using dual UDP probes.
- Implemented simultaneous UDP hole punching with 200ms retry bursts and automatic WebSocket relay fallback for symmetric-to-symmetric NAT scenarios.
- Layered QUIC transport (aioquic) with TLS 1.3 certificate-based authentication and 0-RTT session resumption, achieving 50–100ms handshake savings on reconnection.
- Authored a native C++ extension via pybind11 for high-throughput UDP packet bursting, bypassing Python's GIL to minimize hole-punch latency.
- Implemented QUIC multiplexed streams for control/ping, chat, and binary file transfer, with an HTTP metrics dashboard tracking RTT, connection mode, and 0-RTT status.

[GitHub](#)

MultiModal Document Intelligence Platform

May 2026 – Present

Python · Flask · LangGraph · ChromaDB · Celery · Redis · EasyOCR · React · SQLAlchemy · Docker · AWS S3

- Engineered an end-to-end document intelligence platform with async Celery + Redis ingestion pipeline performing layout-aware PDF parsing (Docling), local OCR (EasyOCR — English, Hindi, Marathi), and bounding-box coordinate storage.
- Implemented a LangGraph multi-node agentic RAG system with dynamic tool routing across ChromaDB vector retrieval, SerpAPI web search, and a sandboxed Python code interpreter — all with strict multi-tenant isolation by user_id and project_id.
- Designed hybrid retrieval combining semantic search (SentenceTransformers all-MiniLM-L6-v2 + ChromaDB) and lexical BM25 search fused via Reciprocal Rank Fusion (RRF), improving retrieval precision over pure vector search.
- Built a citation parser mapping inline references to page-level bounding boxes rendered as overlays on the PDF canvas in React; streamed token-level responses via Server-Sent Events (SSE) for minimal time-to-first-token (TTFT).
- Deployed full stack via Docker Compose with JWT + Redis-based authentication; processes documents entirely offline with zero cloud API cost for OCR, ensuring data privacy for sensitive documents.

[GitHub](#)

File Compression — BWT + MTF + Huffman Coding

Sep – Oct 2024

C++ · Data Structures & Algorithms · Huffman Coding · Suffix Arrays · WebAssembly · React

- Implemented a 3-stage lossless compression pipeline: Burrows-Wheeler Transform (BWT) for character clustering, Move-To-Front (MTF) encoding, and Huffman Coding for entropy-based variable-length encoding.
- Achieved ~80% compression ratio on large text files; designed a custom binary format (.rsk) encoding the Huffman tree and bitstream for faithful lossless decompression.
- Exposed the C++ compression core to a React frontend via WebAssembly (Emscripten), enabling in-browser file compression with a Web Worker for non-blocking UI.

[GitHub](#) | [Live Demo](#)

EDUCATION

B.Tech — Computer Science and Engineering

CGPA: 7.98 / 10.0 | Expected May 2027

COEP Technological University · Pune, Maharashtra

CERTIFICATIONS

IBM Backend Development Professional Certificate	IBM	2026
RAG & Agentic AI Professional Certificate	IBM	2026
Intermediate to Advanced Python	Udemy	2024

LANGUAGES

English — Fluent · Hindi — Fluent · Marathi — Native